

ESP32 GPIO Interrupts, Button Handling, ISR, Queue, and Debounce — Complete Deep Dive

1. Introduction

This document explains:

- GPIO input configuration in ESP-IDF
- GPIO interrupts
- ISR (Interrupt Service Routine)
- Why ISR must be short
- Why Queue is used between ISR and Task
- FreeRTOS task communication
- Button debounce problem
- Software debounce
- Hardware debounce
- Pull-up and Pull-down resistors
- Different GPIO interrupt types
- RTOS task states
- Internal execution flow
- Professional embedded design practices

The goal is not only to make the code work, but to deeply understand WHAT is happening internally.

2. Final Code

```
/*
 * =====
 * ESP32 Button Interrupt Example using Queue + FreeRTOS Task
 * =====
 *
 * This program demonstrates:
 *
 * 1. Configuring a GPIO pin as button input
 * 2. Using GPIO interrupts
 * 3. Using ISR (Interrupt Service Routine)
 * 4. Sending data from ISR to a FreeRTOS task using Queue
 * 5. Software debounce handling
 *
 * -----
 * Hardware Connection
```

```

* -----
*
* GPIO26 ----- BUTTON ----- GND
*
* Internal Pull-up is enabled.
*
* Therefore:
*
* Button NOT pressed:
*     GPIO reads HIGH (1)
*
* Button pressed:
*     GPIO reads LOW (0)
*
* Therefore interrupt type:
*     HIGH -> LOW
*     Falling Edge
*     GPIO_INTR_NEGEDGE
*
* -----
* Why Queue?
* -----
*
* ISR should be extremely short.
*
* ISR should NOT:
*     - print
*     - allocate memory
*     - delay
*     - use mutex
*     - do heavy processing
*
* Therefore ISR only sends an event to queue.
*
* The task performs the actual processing.
*
* -----
* Debounce
* -----
*
* Mechanical buttons bounce electrically.
*
* One physical press may generate:
*     multiple interrupts.
*
* We solve this using timestamp-based software debounce.
*
*/

```

```

#include <stdio.h>

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"

#include "driver/gpio.h"

// =====
// GPIO Number
// =====

#define BUTTON_GPIO GPIO_NUM_26

// =====
// Debounce Variable
// =====

/*
 * Stores last valid button press time.
 *
 * Used to ignore bouncing interrupts.
 */
static uint32_t last_press_time = 0;

// =====
// Queue Handle
// =====

/*
 * Queue is used for communication between:
 *
 * ISR ---> FreeRTOS Task
 *
 * ISR sends GPIO number.
 * Task receives GPIO number.
 */
static QueueHandle_t gpio_evt_queue = NULL;

// =====
// GPIO Interrupt Service Routine (ISR)
// =====

/*
 * IRAM_ATTR:
 *
 * Places ISR in internal RAM.

```

```

*
* Why?
*
* During flash operations, external flash may become unavailable.
* ISRs must still work.
*
* Therefore critical ISR code is stored in IRAM.
*/
static void IRAM_ATTR gpio_isr_handler(void *arg)
{
    /*
     * arg contains the GPIO number.
     *
     * We passed this argument when registering ISR.
     */
    uint32_t gpio_num = (uint32_t)arg;

    /*
     * Send GPIO number to queue.
     *
     * IMPORTANT:
     *
     * We must use:
     *     xQueueSendFromISR()
     *
     * NOT:
     *     xQueueSend()
     *
     * Because ISR context is special.
     */
    xQueueSendFromISR(gpio_evt_queue, &gpio_num, NULL);
}

// =====
// Button Processing Task
// =====

void button_task(void *arg)
{
    uint32_t io_num;

    while (1)
    {
        /*
         * Wait forever for queue event.
         *
         * portMAX_DELAY means:
         *     block indefinitely.

```

```

    *
    * Task consumes almost zero CPU while blocked.
    */
    if (xQueueReceive(gpio_evt_queue,
                      &io_num,
                      portMAX_DELAY))
    {
        /*
         * Current RTOS tick count.
         */
        uint32_t now = xTaskGetTickCount();

        /*
         * Debounce Check
         *
         * Ignore interrupts occurring within 50ms.
         */
        if ((now - last_press_time) > pdMS_TO_TICKS(50))
        {
            printf("VALID BUTTON PRESS\n");

            /*
             * Update last valid press time.
             */
            last_press_time = now;
        }
        else
        {
            printf("BOUNCE IGNORED\n");
        }
    }
}

// =====
// Main Application
// =====

void app_main(void)
{
    // =====
    // GPIO Configuration Structure
    // =====

    gpio_config_t io_conf = {

        /*
         * Bit mask for GPIO selection.

```

```

    *
    * 1ULL means:
    *      unsigned long long (64-bit)
    *
    * Shift 1 to GPIO26 position.
    */
    .pin_bit_mask = (1ULL << BUTTON_GPIO),

    /*
    * GPIO configured as INPUT.
    */
    .mode = GPIO_MODE_INPUT,

    /*
    * Enable internal pull-up resistor.
    *
    * Default state becomes HIGH.
    */
    .pull_up_en = GPIO_PULLUP_ENABLE,

    /*
    * Pull-down disabled.
    */
    .pull_down_en = GPIO_PULLDOWN_DISABLE,

    /*
    * Interrupt type.
    *
    * Trigger interrupt when signal transitions:
    *      HIGH -> LOW
    *
    * Called Falling Edge / Negative Edge.
    */
    .intr_type = GPIO_INTR_NEGEDGE
};

// Apply GPIO configuration
gpio_config(&io_conf);

// =====
// Create Queue
// =====

/*
* Queue length:
*      10 elements
*
* Each element size:

```

```

    *      sizeof(uint32_t)
    */
    gpio_evt_queue = xQueueCreate(10, sizeof(uint32_t));

    // =====
    // Create Task
    // =====

    xTaskCreate(button_task,
                "button_task",
                2048,
                NULL,
                10,
                NULL);

    // =====
    // Install GPIO ISR Service
    // =====

    gpio_install_isr_service(0);

    // =====
    // Attach ISR Handler
    // =====

    gpio_isr_handler_add(BUTTON_GPIO,
                        gpio_isr_handler,
                        (void *)BUTTON_GPIO);

    printf("Waiting for button press...\n");
}

```

3. What is GPIO?

GPIO means:

General Purpose Input Output

These are programmable pins.

GPIO can be configured as:

Mode	Meaning
Input	Read voltage
Output	Drive voltage
Input + Output	Both
Interrupt	Detect changes

4. Why Inputs Float?

If nothing drives a GPIO:

GPIO ---- floating

Voltage becomes undefined.

GPIO may randomly read:

HIGH
LOW
HIGH
LOW

because of:

- EMI noise
- leakage current
- nearby signals

This is called:

Floating Input

5. Pull-Up Resistor

Internal pull-up resistor connects GPIO weakly to:

3.3V

Internally:

3.3V
|
[R]
|
GPIO

Typical ESP32 internal pull-up:

~45k Ω

6. Button Connection using Pull-Up

3.3V
|
[internal pull-up]
|
GPIO26 ---- BUTTON ---- GND

7. Button States

Button NOT pressed

GPIO connected to:

3.3V through resistor

GPIO reads:

HIGH

Button PRESSED

Button connects GPIO directly to:

GND

GPIO reads:

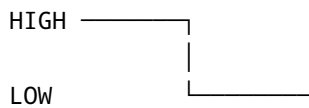
LOW

8. Why GPIO_INTR_NEGEDGE?

Pressing button creates:

HIGH -> LOW

Signal:



This is called:

- Falling Edge
- Negative Edge

Therefore:

GPIO_INTR_NEGEDGE

9. Different GPIO Interrupt Types

Interrupt Type	Meaning
GPIO_INTR_POSEDGE	LOW -> HIGH

Interrupt Type	Meaning
GPIO_INTR_NEGEDGE	HIGH -> LOW
GPIO_INTR_ANYEDGE	Both edges
GPIO_INTR_LOW_LEVEL	While LOW
GPIO_INTR_HIGH_LEVEL	While HIGH
GPIO_INTR_DISABLE	Disable interrupt

10. Positive Edge Example

Using pull-down:

```

3.3V ---- BUTTON ---- GPIO
                        |
                        [R]
                        |
                        GND

```

Default:

LOW

Pressed:

HIGH

Transition:

LOW -> HIGH

Therefore:

GPIO_INTR_POSEDGE

11. What is ISR?

ISR means:

Interrupt Service Routine

When interrupt occurs:

CPU immediately stops current execution and jumps to ISR.

Example:

```
Task running
  ↓
Button press interrupt
  ↓
CPU pauses task
  ↓
ISR executes
  ↓
CPU resumes task
```

12. Why ISR Must Be Short

Interrupts are high priority.

Long ISR creates problems:

- missed interrupts
- system latency
- scheduler delays
- watchdog reset
- unstable system

13. Things NOT Allowed Inside ISR

Avoid:

```
printf()
malloc()
vTaskDelay()
mutex locks
heavy loops
slow processing
```

Because ISR context is special.

14. Why printf() Inside ISR Crashes

printf internally uses:

- locks
- UART driver
- buffering
- mutex

ISR cannot safely use these.

ESP-IDF detects illegal lock access and aborts.

15. Why Queue is Used

Queue safely transfers information from:

```
ISR Context
  ↓
Task Context
```

ISR only sends event.

Task performs processing.

16. Queue Flow

```
Button Press
  ↓
```

```
GPIO Interrupt
  ↓
ISR Executes
  ↓
xQueueSendFromISR()
  ↓
Queue Stores Event
  ↓
Task Wakes Up
  ↓
Task Processes Event
```

17. Why xQueueSendFromISR()?

FreeRTOS provides special ISR-safe APIs.

Normal APIs cannot be used inside ISR.

Normal API	ISR-safe Version
xQueueSend()	xQueueSendFromISR()
xSemaphoreGive()	xSemaphoreGiveFromISR()

18. FreeRTOS Task States

State	Meaning
Running	Currently executing
Ready	Waiting for CPU
Blocked	Waiting for event
Suspended	Paused manually

19. What Happens in xQueueReceive()

```
xQueueReceive(..., portMAX_DELAY)
```

means:

```
Wait forever until queue receives data.
```

Task enters:

```
BLOCKED state
```

Consumes almost zero CPU.

20. Event Driven Programming

Instead of:

```
while(1)
{
    check_button();
}
```

(CPU wasting)

We do:

```
Sleep until interrupt occurs
```

Efficient RTOS design.

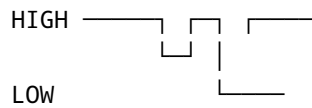
21. What is Button Debounce?

Mechanical buttons are NOT electrically clean.

When pressed:

Contacts physically vibrate.

Signal becomes:



One physical press generates:

multiple interrupts

22. Debounce Problem

Without debounce:

One press may print:

VALID BUTTON PRESS
VALID BUTTON PRESS
VALID BUTTON PRESS

23. Software Debounce

We store:

last_press_time

If interrupts occur within:

50ms

ignore them.

24. Debounce Logic

```
Interrupt at 100ms -> valid  
Interrupt at 102ms -> ignored  
Interrupt at 105ms -> ignored
```

25. Why Release Also Triggers Interrupt Sometimes

Buttons bounce during:

- press
- release

Release signal may become:

```
LOW -> HIGH -> LOW -> HIGH
```

Temporary HIGH->LOW creates another negative edge.

26. Better Debounce Strategy

Professional approach:

```
Interrupt occurs  
↓  
Wait 10-20ms  
↓  
Read stable GPIO level  
↓  
Validate button state
```

27. Hardware Debounce

Using resistor + capacitor.

Example:

```
3.3V
|
[10k]
|
+---- GPIO ---- BUTTON ---- GND
|
[100nF]
|
GND
```

Capacitor smooths fast transitions.

28. RC Time Constant

Formula:

$$\tau = RC$$

Typical values:

Resistor	Capacitor
10k Ω	100nF

29. Why Hardware Debounce Alone is Not Enough

Capacitor improves signal.

But:

- thresholds still exist
- noise still exists
- contacts still bounce

Therefore most products use:

Hardware debounce + Software debounce

30. Why IRAM_ATTR?

ESP32 executes code from external flash.

During some operations:

```
flash temporarily unavailable
```

Interrupts must still work.

Therefore ISR stored in:

```
Internal RAM (IRAM)
```

31. What is GPIO Interrupt Service?

```
gpio_install_isr_service(0);
```

Installs global GPIO interrupt dispatcher.

Without this:

```
GPIO interrupts will not work.
```

32. What Does gpio_isr_handler_add() Do?

```
gpio_isr_handler_add()
```

Associates:

```
GPIO Number -> ISR Function
```

33. Why Use Queue Instead of Global Variable?

Queue advantages:

- thread-safe
- RTOS-safe
- buffering
- event ordering
- multiple events supported

34. Queue vs Semaphore vs Task Notification

Method	Purpose
Queue	Transfer data
Semaphore	Signal event
Task Notification	Fast lightweight signal
Event Group	Multiple event flags

35. Why Semaphore is Better for Simple Button Press

If no data needs transfer:

Queue is overkill.

Semaphore is cleaner.

Because only signaling needed:

Button pressed!

36. Task Notification

Fastest ISR->Task signaling method.

Lower overhead than:

- Queue
- Semaphore

Very commonly used in modern FreeRTOS systems.

37. Why Interrupts Are Better Than Polling

Polling:

```
while(1)
{
    gpio_get_level();
}
```

Problems:

- wastes CPU
- increases power
- inefficient

Interrupts:

CPU sleeps until event occurs

Efficient.

38. Important Embedded Principle

ISR should:

Capture and Defer

Meaning:

Capture event quickly.

Defer heavy processing to task.

This is one of the most important RTOS design principles.

39. Real Industry Practices

Professional systems often use:

Hardware Filter

- + Software Debounce
- + RTOS Signaling
- + State Validation

40. Final Summary

This project teaches:

- GPIO input handling
- Pull-up resistors
- Interrupt edge detection
- ISR execution
- Queue communication
- RTOS task blocking
- Event-driven programming
- Software debounce
- Hardware debounce
- Professional embedded architecture

These are foundational embedded systems concepts used in:

- automotive ECUs
- industrial controllers
- IoT devices
- robotics
- consumer electronics
- medical devices
- RTOS applications